Studio Nx

S1-10a 타임라인 클립 · 연결선 저장 — 함수 설명 합본 (JS 282~287)

2026-10-19

목차

1. ai-notes/javascript/S1-10a - 클립·연결 저장/282_sequencer-devices_clips.md
2. ai-notes/javascript/S1-10a - 클립·연결 저장/283_sequencer-devices_connections.md
3. ai-notes/javascript/S1-10a - 클립·연결 저장/284_sequencer-arrange_clip_ids.md
4. ai-notes/javascript/S1-10a - 클립·연결 저장/285_sequencer_clip_save.md
5. ai-notes/javascript/S1-10a - 클립·연결 저장/286_sequencer_patterns.md
6. ai-notes/javascript/S1-10a - 클립·연결 저장/287_sequencer-roll_clip_notes.md

JavaScript 282 — 클립 색인과 타임라인 클립 그리기 (sequencer-devices.js)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10a 클립·연결·패턴 API: `/full` 클립 색인, 편곡 클립 요소 만들기, MIDI 미리보기, 펼침 칸 다시 채우기 (PRD 02 FR-2196 클립·패턴, FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 273 `sequencer-devices.js` 전체 채우기, 다음 283 Connect 칸

1. 하는 일

`GET /projects/{id}/full` 로 받은 `clips` 와 `patterns` 를 모듈 안에 담아 두고, 트랙을 그릴 때 그 트랙의 클립만 골라 `createClip` 으로 편곡 화면에 붙입니다. MIDI 클립은 노트를 `canvas.sq-clip__preview` 에 작은 막대로 그립니다. 그리고 `/full` 을 다시 받은 뒤 열려 있는 펼침 칸만 다시 채웁니다.

```
GET /projects/{id}/full
  ├─ patterns → indexPatterns → patternsById[pattern_id]
  ├─ clips    → indexClips    → clips 배열
  └─ tracks   → renderTracks
        ↓
  clipsFor(track) — 한 트랙의 클립만 (start_tick 순)
        ↓
  clipElement(track, c) — createClip({ …, clipId, patternId, sampleId })
        ↓ button.sq-clip (.sq-clip--midi | .sq-clip--audio)
  drawClipPreviews() → drawMidiPreview(clip) — canvas 에 노트 막대

applyFullProject / refreshConnections
        ↓
  refreshOpenCells() — .is-open 인 트랙 줄만 ensureLiveCells
```

2. 언제 쓰나

쓰는 곳	
`applyFullProject(data)`	`/full` 의 patterns 와 clips 를 각각 `indexPatterns`, `indexClips` 로 담습니다
`renderTrack` 이 펼침 칸을 만들 때	`clipsFor(track)` 로 그 트랙의 클립을 `clipElement` 로 그립니다
레이아웃이 그려진 뒤	`drawClipPreviews()` 가 모든 MIDI 클립 미리보기를 다시 그립니다
`applyFullProject` · `initDevices` · `refreshConnections`	`refreshOpenCells()` 로 펼쳐 둔 칸만 다시 채웁니다
`sequencer.js` 저장 경로	클립의 `dataset.clipId`, `dataset.patternId`, `dataset.sampleId` 를 읽어 PATCH · DELETE 에 씁니다

3. 코드

```
/** 패턴(노트 포함)을 번호로 담기 (S1-10a) — MIDI 클립 이름 · 미리보기에 씀 */
function indexPatterns(list) {
  (list || []).forEach(function (pattern) { patternsById[pattern.pattern_id] = pattern; });
}

/** 프로젝트의 모든 클립을 담기 (S1-10a FR-2196) */
function indexClips(list) {
  clips = (list || []).slice();
}

/** 한 트랙의 클립 (시작 tick 순) */
function clipsFor(track) {
  return clips.filter(function (c) { return String(c.track_id) === String(track.track_id); })
    .sort(function (a, b) { return a.start_tick - b.start_tick; });
}

/** 서버 클립 한 건 → 편곡 클립 요소 (번호를 달아 두어 저장·삭제가 이어짐) */
function clipElement(track, c) {
  const kind = c.clip_type === 'AUDIO' ? 'audio' : 'midi';
  const el = createClip({
    kind: kind,
    name: clipName(c),
    start: ticksToBars(c.start_tick),
    length: ticksToBars(c.length_tick),
    clipId: c.clip_id,
    patternId: c.pattern_id,
    sampleId: c.sample_id,
  });
  if (c.offset_tick) el.style.setProperty('--o', ticksToBars(c.offset_tick));
  if (kind === 'midi' && c.pattern_id && patternsById[c.pattern_id]) {
    const p = patternsById[c.pattern_id];
    el.__notes = p.notes || [];
    if (p.length_tick) el.__patternBars = Math.max(1, ticksToBars(p.length_tick));
  }
  return el;
}

/** 클립 이름 — 저장된 이름이 없으면 패턴 이름 또는 'Sample #N' */
function clipName(c) {
  if (c.name) return c.name;
  if (c.pattern_id) {
    const p = patternsById[c.pattern_id];
    return p ? p.name : 'Pattern #' + c.pattern_id;
  }
  return 'Sample #' + c.sample_id;
}

/** MIDI 클립 미리보기 다시 그리기 (FR-2196) — __notes 의 노트를 음높이·길이대로 작은 막대로 */
function drawMidiPreview(clip) {
  const notes = clip.__notes;
  const canvas = clip.querySelector('.sq-clip__preview');
  if (!canvas || !notes) return;
  const dpr = window.devicePixelRatio || 1;
  const w = canvas.clientWidth;
```

```
  const h = canvas.clientHeight;
  const pw = Math.max(1, Math.round(w * dpr));
  const ph = Math.max(1, Math.round(h * dpr));
  if (canvas.width !== pw || canvas.height !== ph) { canvas.width = pw; canvas.height = ph; }
  const ctx = canvas.getContext('2d');
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, pw, ph);
  if (w < 2 || h < 2 || !notes.length) return;
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  const ticks = (parseFloat(clip.style.getPropertyValue('--l')) || 1) * TICKS_PER_BAR;
  let lo = Infinity;
  let hi = -Infinity;
  for (let i = 0; i < notes.length; i += 1) { if (notes[i].pitch < lo) lo = notes[i].pitch; if (notes
[i].pitch > hi) hi = notes[i].pitch; }
  if (hi === lo) { lo -= 1; hi += 1; }
  const pad = 0.12;
  const barH = Math.max(1, Math.round(h * 0.14));
  ctx.fillStyle = getComputedStyle(clip).getPropertyValue('--tc').trim() || 'currentColor';
  for (let i = 0; i < notes.length; i += 1) {
    const n = notes[i];
    const x = (n.start_tick / ticks) * w;
    const bw = Math.max(1, (n.length_tick / ticks) * w);
    const y = (1 - (n.pitch - lo) / (hi - lo)) * (1 - pad * 2) + pad;
    ctx.fillRect(x, y * h - barH / 2, Math.min(bw, w - x), barH);
  }
}

/** 레이아웃이 그린 뒤 모든 MIDI 클립 미리보기를 다시 그리기 (캔버스 크기가 붙은 뒤에야 됨) */
function drawClipPreviews() {
  Array.prototype.forEach.call(document.querySelectorAll('#lanes .sq-clip--midi'), drawMidiPreview);
}

/** 지금 펼쳐 둔 트랙 줄(.is-open)의 칸을 다시 채우기 — 줄 DOM 이 아니라 트랙 값으로 부릅니다 */
export function refreshOpenCells() {
  Array.prototype.forEach.call(document.querySelectorAll('#tracks > .sq-track.is-open'), function (row)
{
    const track = trackById(row.dataset.trackId);
    if (track) ensureLiveCells(track);
  });
}
```

문법 풀이

식	뜻
`clips = (list || []).slice()`	서버 배열을 그대로 복사해 모듈 안에 담습니다. 빈 값이면 빈 배열입니다
`String(c.track_id) === String(track.track_id)`	숫자와 글자가 섞여도 비교가 어긋나지 않게 둘 다 글자로 바꿉니다
`ticksToBars(c.start_tick)`	서버의 tick 을 편곡 화면이 쓰는 마디 수로 바꿉니다 (1박 = 960 tick)
`el.__notes = p.notes || []`	노트는 dataset 이 아니라 요소에 붙인 임시 값에 담습니다. 그리는 데만 쓰므로 화면에 노출되지 않습니다
`canvas.width = pw`	캔버스 안쪽 해상도를 기기 픽셀 배율(dpr)만큼 키워 선명하게 그립니다
`ctx.setTransform(dpr, 0, 0, dpr, 0, 0)`	이어서 그리는 좌표는 화면 크기 기준으로 두고, 배율은 한 번에 적용합니다

식	뜻
`row.dataset.trackId` 와 `trackById`	열려 있는 줄의 서버 번호를 읽어 트랙 값을 찾습니다
`Array.prototype.forEach.call(...)`	`querySelectorAll` 결과에 forEach 를 붙이는 전통 표기법입니다

4. 왜 이렇게

선택	이유
클립을 `dataset.clipId`, `dataset.patternId`, `dataset.sampleId` 로 담음	저장(PATCH · DELETE)과 패턴 노트 저장이 같은 요소에서 서버 번호를 읽습니다. 요소를 새로 만들 때마다 번호가 따라다닙니다
클립 이름은 `name`, 패턴 이름, `Sample #N` 순서로 결정	서버에 이름이 없어도 화면에 빈 칸이 보이지 않게 합니다
MIDI 노트를 DOM 이 아니라 canvas 에 그림	노트가 수백 개여도 요소 수가 늘지 않아 스크롤이 가볍습니다
`drawClipPreviews` 를 레이아웃 후에 한 번에 부름	클립을 만드는 시점에는 canvas 크기가 아직 0 입니다. 크기가 붙은 뒤에야 그릴 수 있습니다
`refreshOpenCells` 가 `.is-open` 인 줄만 다시 채움	이전에는 `forEach` 의 순번을 `ensureLiveCells` 에 넘기는 문제(0, 1, 2 가 트랙 번호로 읽힘)가 있어, 열려 있지 않은 칸까지 갱신되거나 트랙을 찾지 못했습니다. 줄의 `dataset.trackId` 로 트랙 값을 직접 찾도록 고쳤습니다
`indexPatterns` 은 `patternsById` 에, `indexClips` 은 배열에	패턴은 번호로 바로 찾고, 클립은 트랙별로 걸러내므로 배열이 편합니다

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 시험용 패턴을 만들고 화면에서 MIDI 클립을 놓은 뒤 새로 불러왔을 때 `data-clip-id` 와 `data-pattern-id` 가 남는지, MIDI 클립 안에 `canvas.sq-clip__preview` 에 노트 막대가 그려지는지, 연결을 저장한 뒤 트랙이 다시 그려져도 펼침 칸이 새 값으로 채워지는지 확인합니다. 시험은 14개 항목 모두 통과했고, 화면 사진은 `docs/screenshots/2026-10-19_s1-10a_clips_connections/` 에 있습니다.

JavaScript 283 — Connect 칸과 연결 저장 (sequencer-devices.js)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10a 클립·연결·패턴 API: Connect 칸 그리기, 연결 저장 · 끄기, 트랙 줄 배지 (PRD 02 FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 282 클립 색인과 타임라인 클립, 다음 284 클립에 서버 번호 달기

1. 하는 일

한 트랙이 다른 트랙으로 소리를 보내는 연결(`track_connections`)을 읽어 두었다가, 펼침 칸의 CONNECT TO ANOTHER 칸에 셀렉트와 번호 칸으로 보여 줍니다. 값을 바꾸면 서버에 저장하고, 끄면 지웁니다. `track_connections` 표는 `UNIQUE(from_track_id)` 이므로 보내는 트랙마다 최대 한 건입니다.

```
GET /full 의 tracks[].connections
        ↓ indexConnections
  connections[from_track_id] = 연결 한 건
  connectionsByTarget[to_track_id] = [연결들]
        ↓
  connectHtml(track) — .sq-route (.sq-route__type · __track · __plugin)
        ↓ wireCells 의 input · change
  refillPluginOptions — 대상 트랙 번호가 바뀌면 이펙트 목록 다시 채움
  updateConnection(acc, track)
   ├ 끄기(None)   → DELETE /tracks/{id}/connection
   └ 켜기(Connected) → PUT  /tracks/{id}/connection { to_track_id, target, to_track_plugin_id? }
        ↓
  refreshConnections → →N · ⇠X 배지, Effects 칸의 sends to, Mixer 칸의 OFF 덮개 다시 그림
```

2. 언제 쓰나

쓰는 곳	
`applyFullProject`	`/full` 의 연결을 `indexConnections` 로 담습니다
`renderCells`	CONNECT TO ANOTHER 칸에 `connectHtml(track)` 을 그립니다
`wireCells` 의 `input`	Track 번호 칸이 바뀌면 `refillPluginOptions` 로 Plugin 목록을 다시 채웁니다
`wireCells` 의 `change`	셀렉트를 바꾸면 `is-live` 를 토글하고, 번호와 Plugin 값을 고르면 `updateConnection` 으로 서버에 저장합니다
`effectsHtml` · `mixerHtml` · `syncTrackSubs`	이펙트 사슬 끝의 `sendtoHtml`, 믹서 칸의 `offHtml`, 트랙 이름의 `connectionBadge` 를 붙입니다

3. 코드

```
/** 나가는 연결을 트랙별로 담기 (S1-10a FR-2197) — GET /full 의 각 트랙 connections 를 폅니다. */
function indexConnections(list) {
  connections = {};
  connectionsByTarget = {};
  (list || []).forEach(function (track) {
    const out = track.connections || [];
    if (out.length) connections[track.track_id] = out[0];
    out.forEach(function (c) {
      if (!connectionsByTarget[c.to_track_id]) connectionsByTarget[c.to_track_id] = [];
      connectionsByTarget[c.to_track_id].push(c);
    });
  });
}

/** 표시 번호(N) → 트랙 (MASTER 제외한 순서 기준) */
function trackByNumber(n) {
  const want = Number(n) - 1;
  if (!isFinite(want)) return null;
  for (let i = 0; i < tracks.length; i += 1) {
    if (tracks[i].track_type !== 'MASTER' && tracks[i].order_index === want) return tracks[i];
  }
  return null;
}

/** Connect to Another 칸 그리기 — 지금 연결 값을 셀렉트에 채웁니다 (None/Connected · Track · Plugin) */
function connectHtml(track) {
  const c = connections[track.track_id] || null;
  const on = !!c;
  return '<div class="sq-route' + (on ? ' is-live' : '') + '">'
    + '<label>Connect</label>'
    + '<select class="sq-route__type" aria-label="Connect">'
    + '<option value="none"' + (on ? '' : ' selected') + '>None</option>'
    + '<option value="connected"' + (on ? ' selected' : '') + '>Connected</option></select>'
    + '<label>Track</label>'
    + '<input class="sq-route__track" type="number" min="1" placeholder="—"'
    + (on ? ' value="' + trackNumberOf(c.to_track_id) + '"' : '') + '>'
    + '<label>Plugin</label>'
    + '<select class="sq-route__plugin"' + (on ? '' : ' disabled') + '>'
    + pluginOptionsHtml(on ? c.to_track_id : null, on ? c : null) + '</select>'
    + '</div>';
}

/** Plugin 셀렉트의 option 들 — 'Instruments (layer)' + 대상 트랙의 이펙트 (고른 것 표시) */
function pluginOptionsHtml(targetId, c) {
  const opts = [{ value: 'instruments', label: 'Instruments (layer)' }];
  const d = targetId ? devices[targetId] : null;
  if (d) {
    d.effects.forEach(function (e) {
      const p = pluginById(e.plugin_id);
      opts.push({ value: String(e.track_plugin_id), label: p ? p.name : 'Plugin #' + e.plugin_id });
    });
  } else if (c && c.to_track_plugin_id) {
    opts.push({ value: String(c.to_track_plugin_id), label: connectionTargetLabel(c) });
  }
```

```
  const picked = c && c.target === 'PLUGIN' && c.to_track_plugin_id ? String(c.to_track_plugin_id) : 'in
struments';
  return opts.map(function (o) {
    return '<option value="' + esc(o.value) + '"' + (o.value === picked ? ' selected' : '') + '>' + esc
(o.label) + '</option>';
  }).join('');
}

/** Track 번호가 바뀌면 그 트랙의 이펙트로 Plugin 칸을 다시 채움 (저장은 change 에서) */
function refillPluginOptions(route, track) {
  const num = route.querySelector('.sq-route__track');
  const plugin = route.querySelector('.sq-route__plugin');
  if (!num || !plugin) return;
  const target = trackByNumber(num.value);
  const conn = connections[track.track_id];
  const c = target && conn && conn.to_track_id === target.track_id ? conn : null;
  plugin.innerHTML = pluginOptionsHtml(target ? target.track_id : null, c);
}

/**
 * Connect to Another 값을 서버에 저장 (FR-2197) — Connected 면 PUT, None 이면 DELETE.
 * 부르는 곳: wireCells 의 change 처리. 저장 뒤 트랙 줄 배지를 다시 그립니다.
 */
export async function updateConnection(acc, track) {
  const route = acc.querySelector('.sq-route');
  if (!route) return;
  const type = route.querySelector('.sq-route__type');
  const num = route.querySelector('.sq-route__track');
  const plugin = route.querySelector('.sq-route__plugin');
  const on = !!type && type.value === 'connected' && !!num && num.value;
  if (!on) {
    try {
      await api.delete('/tracks/' + track.track_id + '/connection');
      delete connections[track.track_id];
      refreshConnections();
      notify({ type: 'connection', trackId: track.track_id, on: false });
    } catch (err) {
      if (!(err instanceof ApiError && err.status === 404)) warn(err);
    }
    return;
  }
  const target = trackByNumber(num.value);
  if (!target) { showToast('No track with that number'); return; }
  const value = plugin ? plugin.value : 'instruments';
  const instruments = !value || value === 'instruments';
  const body = {
    to_track_id: target.track_id,
    target: instruments ? 'INSTRUMENTS' : 'PLUGIN',
  };
  if (!instruments) body.to_track_plugin_id = Number(value);
  try {
    const saved = await api.put('/tracks/' + track.track_id + '/connection', body);
    connections[track.track_id] = saved;
    refreshConnections();
    notify({ type: 'connection', trackId: track.track_id, on: true, toTrackId: target.track_id });
  } catch (err) {
    warn(err);
```

```
    showToast(errText(err));
  }
}

/** 이펙트 칸 끝의 '→ sends to Track N › …' (FR-2197) */
function sendtoHtml(c) {
  const label = 'Track ' + trackNumberOf(c.to_track_id) + ' › ' + connectionTargetLabel(c);
  return '<li class="sq-sendto">→ sends to <b>' + esc(label) + '</b>'
    + '<span>then uses Track ' + trackNumberOf(c.to_track_id) + '’s effects and mixer</span></li>';
}

/** 믹서 칸 위 OFF 덮개 (FR-2197) — 보내는 트랙은 자기 이펙트·믹서 대신 받는 트랙 것을 씁니다 */
function offHtml(c) {
  return '<div class="sq-off"><b>OFF</b>Connected to<br>Track ' + trackNumberOf(c.to_track_id) + ' ›<br
>'
    + esc(connectionTargetLabel(c)) + '</div>';
}
```

문법 풀이

식	뜻
`connections[track.track_id] = out[0]`	`UNIQUE(from_track_id)` 이므로 첫 번째 곧 전부입니다
`isFinite(want)`	`Number('')` 는 0 이 되므로, 숫자가 아니면 여기서 null 을 돌려줍니다
`' sq-route' + (on ? ' is-live' : '')`	연결이 있으면 칸 전체에 `is-live` 를 붙여 셀렉트들을 살아 있게 합니다
`plugin.disabled = !on`	끈 상태에서는 Plugin 칸을 고를 수 없게 막습니다
`target: instruments ? 'INSTRUMENTS' : 'PLUGIN'`	악기 겹(레이어)이면 'INSTRUMENTS', 이펙트 하나이면 'PLUGIN' 입니다
`err.status === 404`	이미 끈 연결을 또 끌 때 서버가 404 를 주어도 화면에는 조용히 반영합니다
`setTimeout(..., 0)` (`wireCells`)	change 직후 DOM 값이 아직 바뀌기 전이므로 다음 틱에 읽어 저장합니다

4. 왜 이렇게

선택	이유
셀렉트 값이 `none` 과 `connected` 두 가지	지금 저장할 수 있는 상태는 끈 것과 건 것뿐입니다. sendto 는 대상 트랙이 이미 다른 곳으로 보내는 경우의 표시이므로 `sendtoHtml` 과 `offHtml` 로만 보여 주고 선택값으로는 두지 않았습니다
하나의 연결만 허용	DB 의 `UNIQUE(from_track_id)` 가 마지막으로 지키므로, 같은 트랙에서 다시 저장하면 `PUT` 이 기존 행을 고치는 방식이 됩니다
Track 칸은 번호, Plugin 칸은 셀렉트	사용자가 트랙 이름을 외우지 않아도 화면의 `1.`, `2.` 번호를 그대로 쓸 수 있습니다
`refillPluginOptions` 는 input 에서, 저장은 change 에서	번호를 칠 때마다 저장하면 서버 요청이 너무 많습니다. 목록만 미리 맞추고 저장은 고침이 끝난 뒤에 합니다
보내는 트랙의 믹서에 OFF 덮개	실제로 믹서를 쓰는 쪽은 받는 트랙입니다. 화면이 실제로 쓰지 않는 볼륨 칸을 그대로 두면 잘못 만질 수 있습니다

선택	이유
`notify({ type: 'connection', … })`	연결도 저장 · 이력의 대상이므로 장치 변경과 같은 알림 경로를 씁니다 (273 문서의 `setDeviceNotifier`)

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 트랙의 CONNECT 칸에서 Connected 를 고르고 Track 번호를 넣은 뒤 페이지를 새로 불러와도 연결이 남는지, Effects 칸 끝에 sends to 문구가 붙는지, 대상 트랙 믹서에 OFF 덮개가 생기는지, None 으로 되돌리면 `DELETE` 후 연결이 사라지는지 확인합니다. 시험은 14개 항목 모두 통과했고, 화면 사진은 `docs/screenshots/2026-10-19_s1-10a_clips_connections/02_connection_saved.png` 에 있습니다.

JavaScript 284 — 클립에 서버 번호 달기 (sequencer-arrange.js)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-10a 클립·연결·패턴 API: 편곡 클립에 clipId · patternId · sampleId 달기 (PRD 02 FR-2196 클립·패턴)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 283 Connect 칸과 연결 저장, 다음 (다음 작업) 시퀀서 저장 경로

1. 하는 일

편곡 화면의 클립 요소는 `createClip` 이 만듭니다. S1-10a 에서는 `createClip` 에 서버 번호 세 가지(`clipId`, `patternId`, `sampleId`)를 받아 넣을 수 있게 했습니다. 왼쪽 목록에서 끌어다 놓기, Ctrl+D 복제, Draw · Paint 로 새로 그리기, 가르기, `sequencer.js` 가 읽는 `templateOf` 까지 이 번호가 따라다니도록 연결했습니다.

```
왼쪽 목록 .sq-file (data-pattern-id / data-sample-id)
   │ dragstart → initListDrop 의 data 에 patternId · sampleId 담기
   ▼
createClip({ kind, name, start, length, clipId?, patternId?, sampleId? })
   ▼
button.sq-clip
   ├─ dataset.clipId     ← 서버 clips 표의 clip_id (저장 뒤 PATCH · DELETE 에 씀)
   ├─ dataset.patternId  ← MIDI 클립이 가리키는 patterns 표 번호
   └─ dataset.sampleId   ← AUDIO 클립이 가리키는 samples 표 번호

Ctrl+D 복제 ─┐
가르기(split) ┼─ templateOf(clip) 또는 dataset 직접 읽기 → patternId · sampleId 이어받기
Draw · Paint ┘
```

2. 언제 쓰나

쓰는 곳	
`clipElement` (`sequencer-devices.js`)	`/full` 클립을 화면에 놓을 때 번호를 함께 넘깁니다
`initListDrop` 의 `dragstart`	목록 항목의 `dataset.patternId`, `dataset.sampleId` 를 끄는 자료에 담습니다
Ctrl+D 복제	선택한 클립의 `dataset.patternId`, `dataset.sampleId` 를 복제본에 그대로 넘깁니다
`templateOf`	Draw · Paint 템플릿과 가르기(split)의 오른쪽 조각이 같은 패턴 · 샘플을 가리키게 합니다
`sequencer.js` 저장 경로	`dataset.clipId` 로 PATCH · DELETE, `dataset.patternId` · `dataset.sampleId` 로 클립 만들기에 씁니다

3. 코드

```
/**
 * 클립 하나 만들기 (S1-06 FR-208) — studioNx.html 의 클립과 같은 모양.
 * 서버에서 온 클립이면 번호를 달아 둡니다 (S1-10a FR-2196) — 저장(PATCH·DELETE) 때 씁니다.
 * @param {{ kind: string, name: string, start: number, length: number,
 *   clipId?: number|string, patternId?: number|string, sampleId?: number|string }} param0
 * @returns {HTMLButtonElement}
 */
export function createClip({ kind, name, start, length, clipId, patternId, sampleId }) {
  const clip = document.createElement('button');
  clip.type = 'button';
  clip.className = `sq-clip sq-clip--${kind}`;
  clip.style.setProperty('--s', start);
  clip.style.setProperty('--l', length);
  if (clipId !== undefined && clipId !== null) clip.dataset.clipId = String(clipId);
  if (patternId !== undefined && patternId !== null) clip.dataset.patternId = String(patternId);
  if (sampleId !== undefined && sampleId !== null) clip.dataset.sampleId = String(sampleId);
  const label = document.createElement('span');
  label.className = 'sq-clip__name';
  label.textContent = `${kind === 'audio' ? '↠' : '𝄜'} ${name}`;
  const preview = document.createElement('canvas');
  preview.className = 'sq-clip__preview';
  clip.append(label, preview);
  return clip;
}

  // initListDrop 안 — 왼쪽 목록을 끌 때
  document.addEventListener('dragstart', (e) => {
    const item = e.target.closest?.('.sq-file[draggable="true"][data-kind]');
    if (!item) return;
    data = {
      kind: item.dataset.kind, name: item.dataset.name, length: parseFloat(item.dataset.bars) || 1, colo
r: item.dataset.color,
      // 서버 번호 — MIDI 는 패턴, AUDIO 는 샘플 (S1-10a FR-2196). 없으면 null
      patternId: item.dataset.patternId || null,
      sampleId: item.dataset.sampleId || null,
    };

  // Ctrl+D 복제
  const clip = createClip({ kind, name, start: newStart, length, patternId: c.dataset.patternId, sampleI
d: c.dataset.sampleId });

/** 클립 → 놓을 것 */
function templateOf(clip) {
  return {
    kind: clip.classList.contains('sq-clip--audio') ? 'audio' : 'midi',
    name: clip.querySelector('.sq-clip__name').textContent.replace(/^[𝄜↠]\s*/, '').trim(),
    length: parseFloat(clip.style.getPropertyValue('--l')) || 1,
    // 서버 번호를 이어받아 복제 · 가르기 · 새로 그리기가 같은 패턴/샘플을 가리키게 (S1-10a)
    patternId: clip.dataset.patternId || null,
    sampleId: clip.dataset.sampleId || null,
  };
}
```

문법 풀이

식	뜻
`clipId !== undefined && clipId !== null`	번호를 안 넘기면 속성을 아예 만들지 않습니다. `dataset` 에 `undefined` 글자가 남지 않습니다
`clip.dataset.clipId = String(clipId)`	`dataset` 은 항상 글자입니다. `sequencer.js` 가 읽은 뒤 `Number(...)` 로 다시 바꿉니다
`item.dataset.patternId || null`	속성이 없으면 `undefined` 이므로 `|| null` 로 바꿔 담습니다
`c.dataset.patternId` (Ctrl+D)	복제본도 원본과 같은 패턴을 가리키게 합니다. 새 패턴을 만들지 않습니다
`clip.classList.contains('sq-clip--audio')`	오디오 · MIDI 구분은 클래스 이름으로 읽습니다
`text.replace(/^[目⇥]\s*/, '')`	이름 칸에 붙은 기호를 떼고 본래 이름만 돌려줍니다

4. 왜 이렇게

선택	이유
번호를 `dataset` 에 담음	클립 요소 하나가 화면과 서버 사이의 연결 고리입니다. 요소를 따라다니므로 복제 · 가르기 · 끌기 어디서 읽어도 같은 번호를 줍니다
`createClip` 이 세 번호를 모두 선택으로 받음	새로 만드는 클립(번호 없음)과 서버에서 온 클립(번호 있음)이 같은 함수를 씁니다
`clipId` 는 복제에 넘기지 않음	복제본은 아직 서버에 없는 새 클립입니다. 원본의 `clipId` 를 물려주면 저장할 때 원본을 고치게 됩니다
`patternId` · `sampleId` 는 복제에 넘김	복제본도 같은 노트 · 같은 소리를 연주해야 합니다 (DB 에서도 두 클립은 같은 패턴을 가리킵니다)
`templateOf` 에 번호 포함	Draw · Paint 로 새로 그린 클립과 가르기의 오른쪽 조각도 같은 패턴 · 샘플을 쓰게 하기 위해서입니다
목록 항목은 `data-pattern-id`, `data-sample-id` 로 시작	목록 항목은 `data-kind` 로 종류를 이미 갖고 있으므로, 번호는 마침표 이름 형식을 그대로 따랐습니다

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 왼쪽 목록에서 MIDI 항목을 끌어 클립을 놓은 뒤, `dataset.patternId` 가 붙어 있는지, 저장하면 `dataset.clipId` 가 붙는지, 새로 불러와도 번호가 남는지 확인합니다. 시험은 14개 항목 모두 통과했고, 화면 사진은 `docs/screenshots/2026-10-19_s1-10a_clips_connections/` 에 있습니다 (`01_clip_created.png`, `03_reloaded.png`).

JavaScript 285 — 클립 저장 이어 주기 (`sequencer.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-10a 클립·연결 저장: 클립 생성/복제/이동/크기 조절/트리밍/슬립/자르기/삭제 및 헬퍼 함수 · 패턴 첫 생성 (FR-2196 클립·패턴, FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 284 (관련), 다음 286 `sequencer.js` 패턴·롤 노트 저장

1. 하는 일

타임라인에서 바뀐 클립을 서버에 바로 이어 저장합니다. `handleChange` 의 `create`·`duplicate`·`move`·`resize`·`trim`·`slip`·`slice`·`delete` 케이스를 통해, MIDI/오디오 클립 각각에 맞게 `POST /tracks/{id}/clips`, `PATCH /clips/{id}`, `DELETE /clips/{id}`, 그리고 필요할 때 `POST /patterns` 를 부릅니다.

```
편곡에서 클립을 놓거나 움직이거나 잘라내기
        ↓
sequencer-arrange.js → onChange({ type: ..., clip, trackId, start, length, right, ... })
        ↓
sequencer.js handleChange
  ├ create · duplicate → saveClipCreate        (오디오: sampleId 필요 / MIDI: 패턴 없으면 POST /pattern
s)
  ├ move · resize · trim · slip → saveClipUpdate (server-numbered clips only: has data-clip-id)
  ├ slice → saveClipUpdate(left) + saveClipCreate(right)
  └ delete → saveClipDelete
        ↓
sequencer-devices.js / 서버
        ↓
markDirty('clip') · clipWarn
```

2. 언제 쓰나

쓰는 곳	
`sequencer-arrange.js` 편곡 드래그	클립을 생성/옮김/크기조절/트리밍/슬립/자르기/삭제할 때 `onChange` 로 알립니다.
`sequencer.js` `handleChange`	위 타입들을 받아 서버에 이어 저장합니다.
`initPatternCommands`/다른 경로	`saveNewPattern` 은 `pattern.new` 명령에서도 씁니다.
디버그/에러	`clipWarn`/`errText` 가 저장 실패를 짧게 보여 줍니다.

3. 코드

3.1 handleChange 의 클립 케이스 (S1-10a)

```
    // 클립 (S1-10a FR-2196) — 화면에서 바뀐 클립을 서버에 이어 저장합니다
    case 'create':
```

```
    case 'duplicate':
      saveClipCreate(change);
      break;
    case 'move':
    case 'resize':
    case 'trim':
    case 'slip':
      saveClipUpdate(change);
      break;
    case 'slice':
      saveClipUpdate(clipGeometry(change.clip));
      saveClipCreate({ clip: change.right, trackId: change.trackId, start: styleNumber(change.right, '--
s'), length: styleNumber(change.right, '--l') });
      break;
    case 'delete':
      saveClipDelete(change);
      break;
```

3.2 헬퍼 함수

```
/** 마디 → tick (1박 = 960, 한 마디 = TICKS_PER_BAR) */
function toTicks(bars) {
  return Math.round((Number(bars) || 0) * TICKS_PER_BAR);
}

/** 클립 요소의 CSS 값(마디) 읽기 — --s 시작 · --l 길이 · --o 오프셋 */
function styleNumber(clip, name) {
  const v = parseFloat(clip.style.getPropertyValue(name));
  return isFinite(v) ? v : null;
}

/** 클립 요소의 지금 위치·길이·오프셋 (마디) */
function clipGeometry(clip) {
  return { clip: clip, start: styleNumber(clip, '--s'), length: styleNumber(clip, '--l'), offset: styleN
umber(clip, '--o') };
}

/** 클립 요소의 이름 (앞의 目/⇥ 글자 지우기) */
function clipElementName(clip) {
  const el = clip.querySelector('.sq-clip__name');
  return el ? el.textContent.replace(/^[目⇥]\s*/, '').trim() : '';
}

/** 클립 요소가 오디오인가 */
function isAudioClip(clip) {
  return clip.classList.contains('sq-clip--audio');
}
```

3.3 saveClipCreate

```
/**
 * 새 클립을 서버에 저장 (FR-2196) — 타임라인에 놓은 클립 · 복제 · 가른 오른쪽 토막.
 * MIDI 클립에 패턴이 없으면(빈 곳에 새로 그을 때) 패턴을 먼저 만들고, 다음 그리기가 재사용하도록
 * Draw 템플릿에 그 패턴을 담습니다.
 * @param {{clip: HTMLElement, trackId: number|string, start: number, length: number}} change
```

```
 */
async function saveClipCreate(change) {
  const clip = change.clip;
  const trackId = change.trackId;
  if (!clip || !trackId || !projectIdOf()) return;
  const audio = isAudioClip(clip);
  const body = {
    clip_type: audio ? 'AUDIO' : 'MIDI',
    name: clipElementName(clip).slice(0, 100) || null,
    start_tick: toTicks(change.start),
    length_tick: Math.max(1, toTicks(change.length)),
    offset_tick: toTicks(styleNumber(clip, '--o') || 0),
  };
  if (audio) {
    const sampleId = clip.dataset.sampleId ? Number(clip.dataset.sampleId) : null;
    if (!sampleId) { showToast('Pick a sample from the browser first'); clip.remove(); return; }
    body.sample_id = sampleId;
  } else {
    let patternId = clip.dataset.patternId ? Number(clip.dataset.patternId) : null;
    if (!patternId) {
      try {
        const pattern = await api.post('/patterns', {
          project_id: Number(projectIdOf()),
          name: body.name || 'Pattern',
          length_tick: body.length_tick,
        });
        patternId = pattern.pattern_id;
        clip.dataset.patternId = String(patternId);
        const tpl = getDrawTemplate();
        setDrawTemplate({ kind: 'midi', name: body.name || tpl.name, length: change.length, patternId: p
atternId });
      } catch (err) { clipWarn(err); return; }
    }
    body.pattern_id = patternId;
  }
  try {
    const saved = await api.post('/tracks/' + encodeURIComponent(String(trackId)) + '/clips', body);
    clip.dataset.clipId = String(saved.clip_id);
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.4 saveClipUpdate

```
/**
 * 클립의 위치·길이·오프셋을 서버에 저장 (FR-2196) — 옮기기 · 길이 조절 · 트리밍 · 내용 밀기.
 * 아직 서버에 없는 클립(번호 없음)이나 데모 클립은 건너뜁니다.
 * @param {{clip: HTMLElement, start?: number, length?: number, offset?: number}} change
 */
async function saveClipUpdate(change) {
  const clip = change.clip;
  if (!clip || !projectIdOf()) return;
  const id = clip.dataset.clipId;
  if (!id) return;
  const body = {};
  if (change.start != null) body.start_tick = toTicks(change.start);
```

```
  if (change.length != null) body.length_tick = Math.max(1, toTicks(change.length));
  if (change.offset != null) body.offset_tick = toTicks(change.offset);
  if (!Object.keys(body).length) return;
  try {
    await api.patch('/clips/' + encodeURIComponent(id), body);
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.5 saveClipDelete

```
/** 클립 지우기를 서버에 반영 (FR-2196) */
async function saveClipDelete(change) {
  const clip = change.clip;
  if (!clip || !projectIdOf()) return;
  const id = clip.dataset.clipId;
  if (!id) return;
  try {
    await api.delete('/clips/' + encodeURIComponent(id));
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.6 에러 알림

```
/** 클립·패턴 저장 실패 알림 (조용한 실패 방지) */
function clipWarn(err) {
  if (err instanceof ApiError && err.status === 401) return;
  showToast(errText(err));
  if (globalThis.console) globalThis.console.warn('[sequencer] clip', err);
}

/** 오류 → 짧은 글자 */
function errText(err) {
  if (err && err.status === 403) return 'You cannot change this project';
  if (err && err.status === 404) return 'That clip or pattern no longer exists';
  return 'Could not save the change — we will try again';
}
```

4. 왜 이렇게

선택	이유
`TICKS_PER_BAR` 를 써서 tick 으로 바꿈	DB·서버는 tick 을 쓰고, 프런트 편곡은 마디 단위(`--s`, `--l`, `--o`)를 씁니다. `PPQ=960` 기준과 일관되게 변환합니다.
MIDI 클립에 `patternId` 가 없으면 `POST /patterns` 먼저	빈 곳에 MIDI 클립을 그으면 곧장 패턴이 필요합니다. 패턴을 미리 만들고 `clip.dataset.patternId`/Draw 템플릿에 저장해 재사용과 롤 노트 저장이 이어지게 합니다.
오디오 클립은 `sampleId` 없으면 경고 후 제거	샘플을 고르지 않은 상태에서 오디오 클립을 놓는 건 서버로 보낼 수 없습니다. 화면상에 남겨두지 않기 위해 제거합니다.
`saveClipUpdate` 는 `data-clip-id` 가 있을 때만	서버에서 온 클립만 PATCH/DELETE 합니다. 데모(예시 화면) 클립은 서버 번호가 없으므로 건드리지 않습니다.

선택	이유
`slice` 는 왼쪽은 업데이트, 오른쪽은 생성	자른 뒤 왼쪽 조각은 원래 클립의 위치·길이만 바뀌므로 `PATCH`, 오른쪽 조각은 새 클립이므로 `POST` 로 이어 저장합니다.
`clipWarn` 이 401 은 무시	`api.js` 의 토큰 재발급 흐름에 맡기기 위해 토스트를 띄우지 않습니다.
offset 도 tick 으로 보냄	`offset_tick` 은 서버가 클립 내부 시작을 가리키는 값입니다. 화면의 `--o`(마디)를 tick 으로 변환해 보냅니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 다음을 확인합니다.

- 빈 타임라인에 MIDI 클립을 놓으면 `POST /patterns` → `POST /tracks/{id}/clips` 순으로 이어지고, 클립에 `data-pattern-id` 와 `data-clip-id` 가 생깁니다.
- 오디오 클립을 놓을 때 샘플을 안 고르면 토스트가 나오고 클립이 제거됩니다.
- 클립을 옮기면 `PATCH /clips/{id}` 로 `start_tick` 이 갱신됩니다.
- 클립을 늘리거나 트리밍/슬립하면 길이나 `offset_tick` 이 `PATCH` 로 보냅니다.
- 클립을 자르면 왼쪽은 `PATCH`, 오른쪽은 `POST` 로 보내집니다.
- 클립을 지우면 `DELETE /clips/{id}` 가 호출되고 화면에서 사라집니다.
- 복제한 클립도 `POST /tracks/{id}/clips` 로 새로 저장됩니다.
- 서버에서 불러온 클립(`data-clip-id` 있는 클립)만 업데이트/삭제됩니다.
- 테스트 14/14 통과. 스크린샷: `docs/screenshots/2026-10-19_s1-10a_clips_connections/`.

JavaScript 286 — 패턴 만들기와 롤 노트 저장 (`sequencer.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-10a 클립·연결 저장: 패턴 생성/미사용 패턴 정리/롤 노트 저장 및 롤→클립 미리보기 동기화 (FR-2196 클립·패턴, FR-2244 패턴 메뉴)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 285 클립 저장 이어 주기, 다음 287 서버 클립 노트 불러오기

1. 하는 일

패턴을 만들고, 롤에서 고친 노트를 서버 패턴에 저장하며, 롤 변경이 편곡 클립 미리보기에 바로 반영되게 합니다. `saveNewPattern` 은 PATTERNS 메뉴의 `+ 새 패턴` 버튼/명령에서 쓰이고, `initPatternCommands` 는 `pattern.new`/`pattern.purge` 명령을 등록합니다. `purgeUnusedPatterns` 는 클립이 참조하지 않는 패턴만 삭제합니다. `initClipPreviews` 는 `sq:roll-notes` 이벤트를 듣고 `clip.__notes` 를 갱신한 뒤 `saveClipNotes` 로 `PUT /patterns/{id}/notes` 를 지연 전송합니다.

```
PATTERNS 메뉴: + 새 패턴 클릭 / 커맨드 팔레트 'pattern.new'
        ↓
initPatternCommands → button.click() 또는 saveNewPattern
        ↓
POST /patterns { project_id, name, length_tick: 4 bars }
        ↓
patternsById 업데이트 · markDirty('pattern')

MIDI 롤에서 노트 추가/이동/길이변경/삭제
        ↓
sequencer-roll.js emit 'sq:roll-notes' { clip, notes }
        ↓
sequencer.js initClipPreviews → setClipNotes(clip, notes) + clip.__notes = notes + saveClipNotes
        ↓
WeakMap 타이머 400ms 후 PUT /patterns/{patternId}/notes (모든 변경을 모아서 보냄)
```

2. 언제 쓰나

쓰는 곳	
`sequencer-ui.js`/메뉴	`pattern.new` 명령으로 새 패턴을 만듭니다.
`sequencer-commands.js` 커맨드 팔레트	`pattern.new`, `pattern.purge` 명령으로 호출됩니다.
`sequencer-roll.js`	노트가 바뀔 때마다 `sq:roll-notes` 를 발행합니다.
편곡 화면	`initClipPreviews` 가 `setClipNotes` 를 불러 MIDI 클립 미리보기를 다시 그립니다.

3. 코드

3.1 saveNewPattern

```
/**
 * '+ 새 패턴' 을 서버에 만들기 (FR-2196) — 목록 항목에 번호를 달아 두어 그 패턴을 끌어 놓으면
 * 클립이 이 패턴을 가리킵니다.
 * @param {{name: string, item?: HTMLElement}} change
 */
async function saveNewPattern(change) {
  if (!projectIdOf()) return;
  const name = (change.name || 'Pattern').slice(0, 100);
  try {
    const pattern = await api.post('/patterns', {
      project_id: Number(projectIdOf()),
      name: name,
      length_tick: TICKS_PER_BAR * 4,
    });
    patternsById[pattern.pattern_id] = pattern;
    if (globalThis.console) globalThis.console.debug('[sequencer] pattern', pattern.pattern_id);
    markDirty('pattern');
  } catch (err) { clipWarn(err); }
}
```

3.2 initPatternCommands

```
/** 패턴 명령 등록 — PATTERNS 메뉴에서 패턴을 만들거나 안 쓰는 패턴을 지웁니다 (FR-2244) */
function initPatternCommands() {
  registerCommand('pattern.new', function () {
    const btn = document.getElementById('pattern-new');
    if (btn) btn.click();
  }, { label: 'New pattern', group: 'patterns' });
  registerCommand('pattern.purge', purgeUnusedPatterns, { label: 'Remove unused patterns', group: 'patte
rns' });
}
```

- `pattern.new`: `#pattern-new` 버튼을 클릭해 기존 UI 흐름을 타게 합니다.
- `pattern.purge`: `purgeUnusedPatterns` 를 실행합니다.

3.3 purgeUnusedPatterns

```
/** 안 쓰는 패턴 지우기 (FR-2196) — 클립이 가리키지 않는 패턴만 서버에서 지웁니다 (쓰는 패턴은 400) */
async function purgeUnusedPatterns() {
  if (!projectIdOf()) { showToast('Open a project first'); return; }
  const ids = Object.keys(patternsById);
  let removed = 0;
  for (let i = 0; i < ids.length; i += 1) {
    try {
      await api.delete('/patterns/' + encodeURIComponent(ids[i]));
      delete patternsById[ids[i]];
      removed += 1;
    } catch (err) {
      if (!(err instanceof ApiError) || err.status !== 400) clipWarn(err);
```

```
    }
  }
  if (removed) {
    applyPatterns(Object.keys(patternsById).map(function (id) { return patternsById[id]; }));
    markDirty('pattern');
    showToast(removed + ' unused pattern' + (removed > 1 ? 's' : '') + ' removed');
  } else {
    showToast('No unused patterns to remove');
  }
}
```

- 쓰는 패턴을 지우려 하면 서버가 400을 돌려줍니다. 이 경우는 경고만 무시하고 다음 패턴으로 넘어갑니다.
- 지워진 패턴은 로컬 `patternsById` 에서도 뺍니다.
- 목록은 `applyPatterns` 로 다시 그립니다.

3.4 롤 노트 저장과 미리보기 동기화

```
/**
 * 피아노롤 노트 변화 → 그 MIDI 클립 미리보기 다시 그리기 (요구사항 1, 2026-10-17)
 * - sequencer-roll.js 가 노트를 바꿀 때마다 'sq:roll-notes' 를 보낸다
 */
function initClipPreviews() {
  document.addEventListener('sq:roll-notes', (e) => {
    const detail = e.detail || {};
    if (!detail.clip) return;
    setClipNotes(detail.clip, detail.notes || []);
    detail.clip.__notes = detail.notes || [];   // 다시 열 때 채울 노트도 갱신
    saveClipNotes(detail.clip, detail.notes || []);
  });
}
initClipPreviews();
```

```
/** MIDI 클립의 노트를 서버 패턴에 저장 (S1-10a FR-2196) — 피아노롤에서 고칠 때 호출 (조금 미룸) */
const noteSaveTimers = new WeakMap();
function saveClipNotes(clip, notes) {
  const patternId = clip.dataset.patternId;
  if (!patternId || !projectIdOf()) return;
  const prev = noteSaveTimers.get(clip);
  if (prev) window.clearTimeout(prev);
  noteSaveTimers.set(clip, window.setTimeout(function () {
    noteSaveTimers.delete(clip);
    api.put('/patterns/' + encodeURIComponent(patternId) + '/notes', {
      notes: notes.map(function (n) {
        return { pitch: n.pitch, start_tick: n.start_tick, length_tick: n.length_tick, velocity: n.veloc
ity || 100 };
      }),
    }).then(function () { markDirty('notes'); }).catch(clipWarn);
  }, 400));
}
```

- `__notes` 에 현재 노트 목록을 저장해, 피아노롤을 다시 열거나 `/full` 후 펼침 칸을 채울 때 재사용합니다.
- `WeakMap` 으로 클립마다 타이머를 관리해, 롤에서 연달아 고치는 동안엔 마지막 변경 400ms 후 한 번만 `PUT /patterns/{id}/notes` 합니다.

- 저장된 노트는 `markDirty('notes')` 로 저장 대기열에 표시됩니다.
- `setClipNotes` 는 `sequencer.js` 에서 `drawClipPreview` 로 편곡 미리보기를 다시 그립니다.

4. 왜 이렇게

선택	이유
`pattern.new` 명령이 `#pattern-new` 버튼을 클릭	UI 와 커맨드 팔레트를 하나의 경로로 묶어, 버튼을 눌러도/단축키·팔레트로 불러도 같은 동작을 보장합니다.
`purgeUnusedPatterns` 가 400 만 무시	클립이 여전히 가리키는 패턴은 삭제할 수 없습니다. 서버의 제약을 그대로 따르면서, 에러 토스트를 띄우지 않고 다음 패턴을 시도합니다.
롤 노트 저장을 400ms 디바운스	노트를 끌거나 길이를 바꾸는 동안 수십 번 이벤트가 올 수 있습니다. 타이핑/드래그 중엔 모아 두었다가 마지막에 한 번만 `PUT` 으로 요청 수를 줄입니다.
`clip.__notes` 를 갱신	282에서 `clipElement` 가 `patternsById` 의 노트를 `el.__notes` 에 담고, 롤도 이 값을 읽어 채웁니다. 롤에서 고친 뒤에도 이 값을 최신화해 다시 열 때와 미리보기가 일치하게 합니다.
`saveClipNotes` 가 `patternId` 없으면 조용히 리턴	데모 클립이나 패턴 없는 미완성 클립일 수 있으므로, 불필요한 호출을 피합니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 관련 항목을 확인합니다.

- PATTERNS 목록에서 `+ 새 패턴` 을 누르면 `POST /patterns` 가 호출되고 새 패턴이 목록에 추가됩니다.
- 커맨드 팔레트에서 `pattern.new` 을 실행하면 같은 동작이 일어납니다.
- 안 쓰는 패턴 정리(`pattern.purge`) 시, 클립이 쓰는 패턴은 건너뛰고 안 쓰는 패턴만 삭제됩니다.
- 피아노롤에서 노트를 추가/이동/삭제하면 `sq:roll-notes` 가 발행되고, 편곡의 MIDI 클립 미리보기에 즉시 반영됩니다.
- 롤에서 노트를 고친 뒤 400ms 이내에 다시 고치면 `PUT /patterns/{id}/notes` 는 마지막 한 번만 전송됩니다.
- 롤을 열고 닫은 뒤 다시 열면 마지막으로 저장된 노트가 그대로 불러와집니다.
- 저장 성공 시 `markDirty('notes')` 가 실행됩니다.

JavaScript 287 — 서버 클립 노트 불러오기 (`sequencer-roll.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-roll.js`
작업	S1-10a 클립·연결 저장: 서버 클립 노트 불러오기, 패턴 길이 반영, 롤→시퀀서 노트 저장 경로 연결 (FR-2196 클립·패턴)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 286 패턴 만들기와 롤 노트 저장, 다음 288 (연결)

1. 하는 일

피아노롤을 MIDI 클립에서 열 때, 서버가 준 `clip.__notes` 와 `clip.__patternBars` 를 읽어 롤에 채웁니다. `openRoll` 은 서버 클립이면 패턴 길이를 `setPatternBars` 로 반영하고, 노트를 `loadClipNotes` 로 tick → 마디 단위로 변환해 배치합니다. `loadClipNotes` 는 기존 노트를 지우고, `placeNote`/`renderVelocity`/`updateSelectedInfo` 로 화면을 갱신합니다. 또한 롤 노트가 바뀌면 `emitRollNotes` 가 `sq:roll-notes` 이벤트를 발행해 `sequencer.js` 의 `initClipPreviews` 가 `setClipNotes`, `clip.__notes` 갱신, `saveClipNotes(clip, notes)` 를 통해 `PUT /patterns/{id}/notes` 로 저장하게 이어집니다.

```
MIDI 클립 더블클릭
        ↓
sequencer.js initTracks / 편곡 → openRoll(clip)
        ↓
openRoll
  ├ clip.__patternBars → setPatternBars (격자·끝 표시에 패턴 길이 반영)
  └ clip.__notes → loadClipNotes
        ↓
tick → bars 변환 (start_tick / TICKS_PER_BAR, length_tick / TICKS_PER_BAR)
        ↓
노트 요소 생성 · placeNote · renderVelocity · updateSelectedInfo
        ↓
롤에서 편집 (추가/이동/리사이즈/삭제)
        ↓
initRoll.onChange → emitRollNotes → CustomEvent 'sq:roll-notes' { clip, notes }
        ↓
sequencer.js initClipPreviews → saveClipNotes → PUT /patterns/{id}/notes (400ms debounce)
```

2. 언제 쓰나

쓰는 곳	
편곡 타임라인	MIDI 클립을 더블 클릭하면 `openRoll(clip)` 이 불립니다.
`sequencer-devices.js` (clipElement)	서버 클립을 그릴 때 `el.__patternBars` 와 `el.__notes` 를 채웁니다.
`sequencer.js`	`sq:roll-notes` 이벤트를 듣고 편곡 미리보기를 갱신하며 노트를 저장합니다.
피아노롤 화면	`loadClipNotes` 가 롤 노트 목록을 다시 채웁니다.

3. 코드

3.1 openRoll: 서버 클립 값 불러오기

```
/** 피아노롤 열기 (아래 칸) — clip 이 있으면 제목을 그 클립으로 */
export function openRoll(clip) {
  const bottom = document.getElementById('bottom');
  bottom.hidden = false;
  bottom.querySelectorAll('.sq-panel').forEach((p) => { p.hidden = p.id !== 'panel-roll'; });
  R.clip = clip || null;
  if (clip) {
    document.getElementById('roll-title').textContent = `🎹 Piano roll — ${clip.querySelector('.sq-clip_
_name').textContent.replace(/^目\s*/, '')}`;
    const track = clip.closest('.sq-track');
    const name = track?.querySelector('.sq-track__name b')?.textContent.trim();
    const inst = track?.querySelector('.sq-track__name small')?.textContent.replace(/^.*?🎹\s*/, '').tri
m();
    if (name) document.getElementById('roll-track').textContent = `${name} · ${inst ?? ''}`;
    // 서버에서 온 MIDI 클립이면 그 패턴의 노트를 롤에 채웁니다 (S1-10a FR-2196)
    if (clip.__patternBars) setPatternBars(clip.__patternBars);
    if (clip.__notes) loadClipNotes(clip.__notes);
  }
  requestAnimationFrame(() => {
    applyBarWidth();
    if (!R.opened) { R.opened = true; scrollToNotes(); }
    syncLane();
    requestDraw();
  });
}
```

- `clip.__patternBars` 가 있으면 `setPatternBars` 로 롤의 패턴 길이를 지정합니다.
- `clip.__notes` 가 있으면 `loadClipNotes` 로 서버 노트를 롤에 채웁니다.
- 제목/트랙명은 기존 클립 이름/트랙 정보를 읽어 표시합니다.

3.2 setPatternBars

```
/** 패턴 길이(마디)를 반영 — 격자·끝 표시가 패턴 길이에 맞춥니다 (S1-10a) */
function setPatternBars(bars) {
  if (!R) return;
  const n = Math.max(1, Math.round(bars));
  R.patternBars = n;
  R.roll.style.setProperty('--roll-pattern-bars', n);
  requestDraw();
}
```

- 패턴 길이를 롤 상태에 저장하고 CSS 변수 `--roll-pattern-bars` 에 반영합니다.
- 격자/끝 표시가 해당 길이에 맞춰 보이도록 `requestDraw` 를 호출합니다.

3.3 loadClipNotes

```
/**
 * 서버 패턴의 노트를 롤에 채우기 (S1-10a FR-2196) — 지금 있는 노트(반투명 안내 노트는 남김)를 지우고,
```

```
 * 노트를 tick → 마디로 바꿔 놓습니다. 여기서 고치면 'sq:roll-notes' 로 알려 sequencer.js 가 저장합니다.
 * @param {object[]} notes  {pitch, start_tick, length_tick, velocity}
 */
function loadClipNotes(notes) {
  if (!R) return;
  R.notes.querySelectorAll('.sq-note:not(.is-ghost)').forEach((n) => n.remove());
  const frag = document.createDocumentFragment();
  (notes || []).forEach((nt) => {
    const el = document.createElement('span');
    el.className = 'sq-note';
    el.dataset.pitch = String(nt.pitch);
    el.dataset.start = String(round((nt.start_tick || 0) / TICKS_PER_BAR));
    const len = round((nt.length_tick || TICKS_PER_BAR / 4) / TICKS_PER_BAR);
    el.dataset.length = String(len > 0 ? len : 0.25);
    el.dataset.vel = String(nt.velocity || 100);
    placeNote(el);
    frag.append(el);
  });
  R.notes.append(frag);
  renderVelocity();
  updateSelectedInfo();
}
```

- 서버의 `start_tick`, `length_tick` 을 `TICKS_PER_BAR` 로 나눠 마디 단위(`dataset.start`, `dataset.length`)로 변환합니다.
- 안내용 반투명 노트(`.is-ghost`)는 남겨둡니다.
- `placeNote` 로 위치/길이/음높이 정보를 CSS 변수에 반영하고, `renderVelocity`/`updateSelectedInfo` 로 보조 영역을 갱신합니다.

3.4 롤 노트 발행 (emitRollNotes)

```
/** 지금 피아노롤의 노트 전부를 저장용 모양(tick 단위)으로 모은다 (요구사항 1, 2026-10-17) */
function rollNotes() {
  return [...R.notes.querySelectorAll('.sq-note:not(.is-ghost)')].map((n) => ({
    pitch: Number(n.dataset.pitch),
    start_tick: Math.round(Number(n.dataset.start) * TICKS_PER_BAR),
    length_tick: Math.round(Number(n.dataset.length) * TICKS_PER_BAR),
    velocity: Number(n.dataset.vel) || 100,
  }));
}

/** 노트가 바뀔 때마다 알린다 — sequencer.js 가 듣고 그 MIDI 클립의 미리보기를 다시 그린다 */
function emitRollNotes() {
  if (!R || !R.clip) return;
  document.dispatchEvent(new CustomEvent('sq:roll-notes', {
    detail: { clip: R.clip, notes: rollNotes(), bars: R.patternBars },
  }));
}
```

- 롤에 있는 일반 노트를 tick 단위로 다시 변환해 서버 형식으로 모읍니다.
- `sq:roll-notes` 커스텀 이벤트로 발행해 `sequencer.js` 가 저장과 미리보기 갱신을 이어 받습니다.
- 이벤트 디테일에 `clip` 을 포함해 어떤 클립의 패턴인지 정확히 식별합니다.

3.5 onChange 에 이벤트 발행 연결

```
// 변화 알림 — 사용자 onChange 뒤에 편곡의 클립 미리보기가 다시 그려지도록 알린다 (요구사항, 2026-10-17)
const userChange = onChange;
R.onChange = (change) => {
  if (typeof userChange === 'function') userChange(change);
  emitRollNotes();
};
```

- 피아노롤에서 노트가 추가/이동/리사이즈/삭제될 때마다 `onChange` 가 호출되고, 그 직후 `emitRollNotes` 로 현재 노트 목록을 발행합니다.
- 이렇게 해서 롤 편집 → 클립 미리보기 → 서버 저장까지 한 줄의 경로로 이어집니다.

4. 왜 이렇게

선택	이유
`clip.__notes` 와 `clip.__patternBars` 를 DOM 요소에 보관	서버 클립 정보를 편집창(롤)을 열 때 즉시 불러와 사용할 수 있게 합니다. `sequencer-devices.js` 의 `clipElement` 이 서버 데이터에서 이 값을 채우고, 롤은 그것만 읽습니다.
ghost 노트는 로드 시 지우지 않음	안내용 노트(`.is-ghost`)는 편집 보조용이므로 서버 노트와 겹치지 않게 일반 노트만 제거합니다.
tick ↔ bars 변환을 명시적으로 함	서버는 tick 단위, 롤은 마디 단위로 작업합니다. 로드 시에는 tick → bars, 저장 시에는 `rollNotes()` 가 bars → tick 으로 되돌립니다. 이 변환은 `TICKS_PER_BAR` (960)에 의존합니다.
`emitRollNotes` 가 변경 직후 호출	롤 노트 편집은 드래그 중에도 여러 번 일어나지만, `initRoll` 내부의 편집 경로에서 노트가 화면에 배치된 뒤 `onChange` → `emitRollNotes` 순으로 호출되도록 연결해, 미리보기와 저장 타이밍을 일관되게 만듭니다.
`patternBars` 를 CSS 변수로 노출	롤의 격자와 끝 마커가 패턴 길이에 맞게 그려지도록 CSS에서 참조할 수 있게 합니다.
클립 제목/트랙명을 롤 헤더에 표시	어떤 클립의 패턴을 편집하는지 사용자가 바로 알 수 있게 합니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 항목을 확인합니다.

- 서버에서 불러온 MIDI 클립을 더블 클릭하면 피아노롤이 열리고, 그 클립의 노트가 롤에 정확히 표시됩니다.
- 패턴 길이가 긴 클립은 `setPatternBars` 로 롤 길이가 맞춰집니다.
- 롤에서 노트를 추가하면 `sq:roll-notes` 이벤트가 발행되고, 편곡 화면의 해당 MIDI 클립 미리보기에 막대가 즉시 그려집니다.
- 롤에서 노트를 이동/리사이즈/삭제해도 미리보기와 저장(`PUT /patterns/{id}/notes`)이 이어집니다.
- 롤을 닫았다가 같은 클립을 다시 열면, 마지막으로 저장된 노트 상태가 그대로 불러와집니다.
- 테스트 14/14 통과. 스크린샷: `docs/screenshots/2026-10-19_s1-10a_clips_connections/`.